



Chapter 6: Formal Relational Query Languages

Database System Concepts, 6th Ed.

©Silberschatz, Korth and Sudarshan

See www.db-book.com for conditions on re-use



Chapter 6: Formal Relational Query Languages

- Relational Algebra
- Tuple Relational Calculus
- Domain Relational Calculus



Relational Algebra

- Procedural language
- Six basic operators
 - select: σ
 - project: Π
 - union: $\cup$
 - set difference: $-$
 - Cartesian product: $\times$
 - rename: ρ
- The operators take one or two relations as inputs and produce a new relation as a result.



Select Operation – Example

- Relation r

A	B	C	D
α	α	1	7
α	β	5	7
β	β	12	3
β	β	23	10

- $\sigma_{A=B \wedge D > 5}(r)$

A	B	C	D
α	α	1	7
β	β	23	10



Select Operation

- Notation: $\sigma_p(r)$
- p is called the **selection predicate**
- Defined as:

$$\sigma_p(r) = \{t \mid t \in r \text{ and } p(t)\}$$

Where p is a formula in propositional calculus consisting of **terms** connected by : $\wedge$ (**and**), $\vee$ (**or**), $\neg$ (**not**)

Each **term** is one of:

$\langle \text{attribute} \rangle \quad op \quad \langle \text{attribute} \rangle$ or $\langle \text{constant} \rangle$

where op is one of: $=, \neq, >, \geq, <, \leq$

- Example of selection:

$$\sigma_{dept_name="Physics"}(instructor)$$



Project Operation – Example

- Relation r :

A	B	C
α	10	1
α	20	1
β	30	1
β	40	2

- $\Pi_{A,C}(r)$

A	C
α	1
α	1
β	1
β	2

 $=$

A	C
α	1
β	1
β	2



Project Operation

- Notation:

$$\Pi_{A_1, A_2, \dots, A_k}(r)$$

where A_1, A_2 are attribute names and r is a relation name.

- The result is defined as the relation of k columns obtained by erasing the columns that are not listed
- Duplicate rows removed from result, since relations are sets
- Example: To eliminate the *dept_name* attribute of *instructor*

$$\Pi_{ID, name, salary}(instructor)$$



Union Operation – Example

- Relations r , s :

A	B
α	1
α	2
β	1

r

A	B
α	2
β	3

s

- $r \cup s$:

A	B
α	1
α	2
β	1
β	3



Union Operation

- Notation: $r \cup s$
- Defined as:

$$r \cup s = \{t \mid t \in r \text{ or } t \in s\}$$

- For $r \cup s$ to be valid.
 1. r, s must have the same **arity** (same number of attributes)
 2. The attribute domains must be **compatible** (example: 2nd column of r deals with the same type of values as does the 2nd column of s)
- Example: to find all courses taught in the Fall 2009 semester, or in the Spring 2010 semester, or in both

$$\Pi_{\text{course_id}} (\sigma_{\text{semester}=\text{"Fall"} \wedge \text{year}=2009} (\text{section})) \cup \Pi_{\text{course_id}} (\sigma_{\text{semester}=\text{"Spring"} \wedge \text{year}=2010} (\text{section}))$$





Set difference of two relations

- Relations r, s :

A	B
α	1
α	2
β	1

r

A	B
α	2
β	3

s

- $r - s$:

A	B
α	1
β	1



Set Difference Operation

- Notation $r - s$
- Defined as:

$$r - s = \{t \mid t \in r \text{ and } t \notin s\}$$

- Set differences must be taken between **compatible** relations.
 - r and s must have the **same** arity
 - attribute domains of r and s must be compatible
- Example: to find all courses taught in the Fall 2009 semester, but not in the Spring 2010 semester

$$\Pi_{course_id} (\sigma_{semester="Fall" \wedge year=2009} (section)) - \Pi_{course_id} (\sigma_{semester="Spring" \wedge year=2010} (section))$$





Cartesian-Product Operation – Example

■ Relations r , s :

A	B
α	1
β	2

r

C	D	E
α	10	a
β	10	a
β	20	b
γ	10	b

s

■ $r \times s$:

A	B	C	D	E
α	1	α	10	a
α	1	β	10	a
α	1	β	20	b
α	1	γ	10	b
β	2	α	10	a
β	2	β	10	a
β	2	β	20	b
β	2	γ	10	b



Cartesian-Product Operation

- Notation $r \times s$
- Defined as:

$$r \times s = \{t \ q \mid t \in r \textbf{ and } q \in s\}$$

- Assume that attributes of $r(R)$ and $s(S)$ are disjoint. (That is, $R \cap S = \emptyset$).
- If attributes of $r(R)$ and $s(S)$ are not disjoint, then renaming must be used.



Composition of Operations

- Can build expressions using multiple operations
- Example: $\sigma_{A=C}(r \times s)$

■ $r \times s$

A	B	C	D	E
α	1	α	10	a
α	1	β	10	a
α	1	β	20	b
α	1	γ	10	b
β	2	α	10	a
β	2	β	10	a
β	2	β	20	b
β	2	γ	10	b

■ $\sigma_{A=C}(r \times s)$

A	B	C	D	E
α	1	α	10	a
β	2	β	10	a
β	2	β	20	b



Rename Operation

- Allows us to name, and therefore to refer to, the results of relational-algebra expressions.
- Allows us to refer to a relation by more than one name.
- Example:

$$\rho_x(E)$$

returns the expression E under the name X

- If a relational-algebra expression E has arity n , then

$$\rho_x(A_1, A_2, \dots, A_n)(E)$$

returns the result of expression E under the name X , and with the attributes renamed to $A_1, A_2, \dots, A_n$.





Example Query

- Find the largest salary in the university
 - Step 1: find instructor salaries that are less than some other instructor salary (i.e. not maximum)
 - using a copy of *instructor* under a new name *d*
 - ▶ $\Pi_{instructor.salary} (\sigma_{instructor.salary < d.salary} (instructor \times \rho_d (instructor)))$
 - Step 2: Find the largest salary
 - ▶ $\Pi_{salary} (instructor) - \Pi_{instructor.salary} (\sigma_{instructor.salary < d.salary} (instructor \times \rho_d (instructor)))$



Example Queries

- Find the names of all instructors in the Physics department, along with the *course_id* of all courses they have taught

- Query 1

$$\Pi_{instructor.ID, course_id} (\sigma_{dept_name = \text{"Physics"}} (\sigma_{instructor.ID = teaches.ID} (instructor \times teaches)))$$

- Query 2

$$\Pi_{instructor.ID, course_id} (\sigma_{instructor.ID = teaches.ID} (\sigma_{dept_name = \text{"Physics"}} (instructor \times teaches)))$$



Formal Definition

- A basic expression in the relational algebra consists of either one of the following:
 - A relation in the database
 - A constant relation
- Let E_1 and E_2 be relational-algebra expressions; the following are all relational-algebra expressions:
 - $E_1 \cup E_2$
 - $E_1 - E_2$
 - $E_1 \times E_2$
 - $\sigma_P(E_1)$, P is a predicate on attributes in E_1
 - $\Pi_S(E_1)$, S is a list consisting of some of the attributes in E_1
 - $\rho_x(E_1)$, x is the new name for the result of E_1



Additional Operations

We define additional operations that do not add any power to the relational algebra, but that simplify common queries.

- Set intersection
- Natural join
- Assignment
- Outer join



Set-Intersection Operation

- Notation: $r \cap s$
- Defined as:
- $r \cap s = \{ t \mid t \in r \textbf{ and } t \in s \}$
- Assume:
 - r, s have the *same arity*
 - attributes of r and s are compatible
- Note: $r \cap s = r - (r - s)$



Set-Intersection Operation – Example

■ Relation r, s :

A	B
α	1
α	2
β	1

r

A	B
α	2
β	3

s

■ $r \cap s$

A	B
α	2



Natural-Join Operation

- Notation: $r \bowtie s$
- Let r and s be relations on schemas R and S respectively.
Then, $r \bowtie s$ is a relation on schema $R \cup S$ obtained as follows:
 - Consider each pair of tuples t_r from r and t_s from s .
 - If t_r and t_s have the same value on each of the attributes in $R \cap S$, add a tuple t to the result, where
 - ▶ t has the same value as t_r on r
 - ▶ t has the same value as t_s on s
- Example:
 $R = (A, B, C, D)$
 $S = (E, B, D)$
 - Result schema = (A, B, C, D, E)
 - $r \bowtie s$ is defined as:

$$\Pi_{r.A, r.B, r.C, r.D, s.E} (\sigma_{r.B = s.B \wedge r.D = s.D} (r \times s))$$



Natural Join Example

- Relations r, s :

A	B	C	D
α	1	α	a
β	2	γ	a
γ	4	β	b
α	1	γ	a
δ	2	β	b

r

B	D	E
1	a	α
3	a	β
1	a	γ
2	b	δ
3	b	ϵ

s

- $r \bowtie s$

A	B	C	D	E
α	1	α	a	α
α	1	α	a	γ
α	1	γ	a	α
α	1	γ	a	γ
δ	2	β	b	δ



Natural Join and Theta Join

- Find the names of all instructors in the Comp. Sci. department together with the course titles of all the courses that the instructors teach
 - $\Pi_{name, title} (\sigma_{dept_name="Comp. Sci."} (instructor \bowtie teaches \bowtie course))$
- Natural join is associative
 - $(instructor \bowtie teaches) \bowtie course$ is equivalent to $instructor \bowtie (teaches \bowtie course)$
- Natural join is commutative
 - $instruct \bowtie teaches$ is equivalent to $teaches \bowtie instructor$
- The **theta join** operation $r \bowtie_{\theta} s$ is defined as
 - $r \bowtie_{\theta} s = \sigma_{\theta} (r \times s)$



Assignment Operation

- The assignment operation ($\leftarrow$) provides a convenient way to express complex queries.
 - Write query as a sequential program consisting of
 - ▶ a series of assignments
 - ▶ followed by an expression whose value is displayed as a result of the query.
 - Assignment must always be made to a temporary relation variable.



Outer Join

- An extension of the join operation that avoids loss of information.
- Computes the join and then adds tuples from one relation that does not match tuples in the other relation to the result of the join.
- Uses *null* values:
 - *null* signifies that the value is unknown or does not exist
 - All comparisons involving *null* are (roughly speaking) **false** by definition.
 - ▶ We shall study precise meaning of comparisons with nulls later



Outer Join – Example

■ Relation *instructor1*

<i>ID</i>	<i>name</i>	<i>dept_name</i>
10101	Srinivasan	Comp. Sci.
12121	Wu	Finance
15151	Mozart	Music

■ Relation *teaches1*

<i>ID</i>	<i>course_id</i>
10101	CS-101
12121	FIN-201
76766	BIO-101



Outer Join – Example

■ Join

instructor ⋈ *teaches*

<i>ID</i>	<i>name</i>	<i>dept_name</i>	<i>course_id</i>
10101	Srinivasan	Comp. Sci.	CS-101
12121	Wu	Finance	FIN-201

■ Left Outer Join

instructor ⋈_L *teaches*

<i>ID</i>	<i>name</i>	<i>dept_name</i>	<i>course_id</i>
10101	Srinivasan	Comp. Sci.	CS-101
12121	Wu	Finance	FIN-201
15151	Mozart	Music	<i>null</i>



Outer Join – Example

■ Right Outer Join

instructor ⋈_r *teaches*

<i>ID</i>	<i>name</i>	<i>dept_name</i>	<i>course_id</i>
10101	Srinivasan	Comp. Sci.	CS-101
12121	Wu	Finance	FIN-201
76766	null	null	BIO-101

■ Full Outer Join

instructor ⋈_f *teaches*

<i>ID</i>	<i>name</i>	<i>dept_name</i>	<i>course_id</i>
10101	Srinivasan	Comp. Sci.	CS-101
12121	Wu	Finance	FIN-201
15151	Mozart	Music	<i>null</i>
76766	null	null	BIO-101



Outer Join using Joins

- Outer join can be expressed using basic operations

- e.g. $r \bowtie s$ can be written as

$$(r \bowtie s) \cup (r - \Pi_R(r \bowtie s) \times \{null, ..., null\})$$



Null Values

- It is possible for tuples to have a null value, denoted by *null*, for some of their attributes
- *null* signifies an unknown value or that a value does not exist.
- The result of any arithmetic expression involving *null* is *null*.
- Aggregate functions simply ignore null values (as in SQL)
- For duplicate elimination and grouping, null is treated like any other value, and two nulls are assumed to be the same (as in SQL)



Null Values

- Comparisons with null values return the special truth value: *unknown*
 - If *false* was used instead of *unknown*, then $\text{not } (A < 5)$ would not be equivalent to $A \geq 5$
- Three-valued logic using the truth value *unknown*:
 - OR: $(\text{unknown or true}) = \text{true}$,
 $(\text{unknown or false}) = \text{unknown}$
 $(\text{unknown or unknown}) = \text{unknown}$
 - AND: $(\text{true and unknown}) = \text{unknown}$,
 $(\text{false and unknown}) = \text{false}$,
 $(\text{unknown and unknown}) = \text{unknown}$
 - NOT: $(\text{not unknown}) = \text{unknown}$
 - In SQL “*P* is **unknown**” evaluates to true if predicate *P* evaluates to *unknown*
- Result of select predicate is treated as *false* if it evaluates to *unknown*



Division Operator

- Given relations $r(R)$ and $s(S)$, such that $S \subset R$, $r \div s$ is the largest relation $t(R-S)$ such that
$$t \times s \subseteq r$$
- E.g. let $r(ID, course_id) = \Pi_{ID, course_id} (takes)$ and
$$s(course_id) = \Pi_{course_id} (\sigma_{dept_name="Biology"}(course))$$
then $r \div s$ gives us students who have taken all courses in the Biology department
- Can write $r \div s$ as

$$temp1 \leftarrow \Pi_{R-S} (r)$$

$$temp2 \leftarrow \Pi_{R-S} ((temp1 \times s) - \Pi_{R-S,S} (r))$$

$$result = temp1 - temp2$$

- The result to the right of the $\leftarrow$ is assigned to the relation variable on the left of the $\leftarrow$.
- May use variable in subsequent expressions.



Extended Relational-Algebra-Operations

- Generalized Projection
- Aggregate Functions



Generalized Projection

- Extends the projection operation by allowing arithmetic functions to be used in the projection list.

$$\Pi_{F_1, F_2, \dots, F_n}(E)$$

- E is any relational-algebra expression
- Each of $F_1, F_2, \dots, F_n$ are arithmetic expressions involving constants and attributes in the schema of E .
- Given relation *instructor*(*ID*, *name*, *dept_name*, *salary*) where *salary* is annual salary, get the same information but with monthly salary

$$\Pi_{ID, name, dept_name, salary/12} (instructor)$$



Aggregate Functions and Operations

- **Aggregation function** takes a collection of values and returns a single value as a result.

avg: average value **min**:
minimum value **max**:
maximum value **sum**:
sum of values **count**:
number of values

- **Aggregate operation** in relational algebra

$$G_1, G_2, K, G_n \quad \mathcal{G}_{F_1(A_1), F_2(A_2), K, F_n(A_n)}(E)$$

E is any relational-algebra expression

- $G_1, G_2, \dots, G_n$ is a list of attributes on which to group (can be empty)
- Each F_i is an aggregate function
- Each A_i is an attribute name

- Note: Some books/articles use γ instead of $\mathcal{G}$ (Calligraphic G)



Aggregate Operation – Example

- Relation r :

A	B	C
α	α	7
α	β	7
β	β	3
β	β	10

- $G_{\text{sum}(c)}(r)$

$\text{sum}(c)$
27



Aggregate Operation – Example

- Find the average salary in each department

dept_name $G_{avg}(salary)$ (*instructor*)

<i>ID</i>	<i>name</i>	<i>dept_name</i>	<i>salary</i>
76766	Crick	Biology	72000
45565	Katz	Comp. Sci.	75000
10101	Srinivasan	Comp. Sci.	65000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000
12121	Wu	Finance	90000
76543	Singh	Finance	80000
32343	El Said	History	60000
58583	Califieri	History	62000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
22222	Einstein	Physics	95000

<i>dept_name</i>	<i>avg_salary</i>
Biology	72000
Comp. Sci.	77333
Elec. Eng.	80000
Finance	85000
History	61000
Music	40000
Physics	91000



Aggregate Functions (Cont.)

- Result of aggregation does not have a name
 - Can use rename operation to give it a name
 - For convenience, we permit renaming as part of aggregate operation

```
dept_name Gavg(salary) as avg_sal (instructor)
```



Modification of the Database

- The content of the database may be modified using the following operations:
 - Deletion
 - Insertion
 - Updating
- All these operations can be expressed using the assignment operator



Multiset Relational Algebra

- Pure relational algebra removes all duplicates
 - e.g. after projection
- Multiset relational algebra retains duplicates, to match SQL semantics
 - SQL duplicate retention was initially for efficiency, but is now a feature
- Multiset relational algebra defined as follows
 - selection: has as many duplicates of a tuple as in the input, if the tuple satisfies the selection
 - projection: one tuple per input tuple, even if it is a duplicate
 - cross product: If there are m copies of $t1$ in r , and n copies of $t2$ in s , there are $m \times n$ copies of $t1.t2$ in $r \times s$
 - Other operators similarly defined
 - ▶ E.g. union: $m + n$ copies, intersection: $\min(m, n)$ copies
difference: $\min(0, m - n)$ copies



SQL and Relational Algebra

- **select** $A1, A2, \dots, An$
from $r1, r2, \dots, rm$
where P

is equivalent to the following expression in multiset relational algebra

$$\prod_{A1, \dots, An} (\sigma_P (r1 \times r2 \times \dots \times rm))$$

- **select** $A1, A2, \text{sum}(A3)$
from $r1, r2, \dots, rm$
where P
group by $A1, A2$

is equivalent to the following expression in multiset relational algebra

$$A1, A2 \text{ } \mathcal{G} \text{sum}(A3) (\sigma_P (r1 \times r2 \times \dots \times rm)))$$





SQL and Relational Algebra

- More generally, the non-aggregated attributes in the **select** clause may be a subset of the **group by** attributes, in which case the equivalence is as follows:

```
select A1, sum(A3)
from   r1, r2, ..., rm
where P
group by A1, A2
```

is equivalent to the following expression in multiset relational algebra

$$\Pi_{A1, \text{sum}A3} (A1, A2 \text{ } \mathcal{G} \text{ sum}(A3) \text{ as sum}A3 (\sigma_P (r1 \times r2 \times \dots \times rm)))$$





Tuple Relational Calculus



Tuple Relational Calculus

- A nonprocedural query language, where each query is of the form

$$\{t \mid P(t)\}$$

- It is the set of all tuples t such that predicate P is true for t
- t is a *tuple variable*, $t[A]$ denotes the value of tuple t on attribute A
- $t \in r$ denotes that tuple t is in relation r
- P is a *formula* similar to that of the predicate calculus



Predicate Calculus Formula

1. Set of attributes and constants
2. Set of comparison operators: (e.g., $<$, $\leq$, $=$, $\neq$, $>$, $\geq$)
3. Set of connectives: and ($\wedge$), or ($\vee$), not ($\neg$)
4. Implication ($\Rightarrow$): $x \Rightarrow y$, if x is true, then y is true

$$x \Rightarrow y \equiv \neg x \vee y$$

5. Set of quantifiers:

- ▶ $\exists t \in r (Q(t)) \equiv$ "there exists" a tuple t in relation r such that predicate $Q(t)$ is true
- ▶ $\forall t \in r (Q(t)) \equiv Q$ is true "for all" tuples t in relation r



Example Queries

- Find the *ID*, *name*, *dept_name*, *salary* for instructors whose salary is greater than \$80,000

$$\{t \mid t \in instructor \wedge t[salary] > 80000\}$$

- As in the previous query, but output only the *ID* attribute value

$$\{t \mid \exists s \in instructor (t[ID] = s[ID] \wedge s[salary] > 80000)\}$$

Notice that a relation on schema (*ID*) is implicitly defined by the query



Example Queries

- Find the names of all instructors whose department is in the Watson building

$$\{t \mid \exists s \in \text{instructor } (t[\text{name}] = s[\text{name}] \wedge \exists u \in \text{department } (u[\text{dept_name}] = s[\text{dept_name}] \wedge u[\text{building}] = \text{"Watson"}))\}$$

- Find the set of all courses taught in the Fall 2009 semester, or in the Spring 2010 semester, or both

$$\{t \mid \exists s \in \text{section } (t[\text{course_id}] = s[\text{course_id}] \wedge s[\text{semester}] = \text{"Fall"} \wedge s[\text{year}] = 2009 \vee \exists u \in \text{section } (t[\text{course_id}] = u[\text{course_id}] \wedge u[\text{semester}] = \text{"Spring"} \wedge u[\text{year}] = 2010))\}$$



Example Queries

- Find the set of all courses taught in the Fall 2009 semester, and in the Spring 2010 semester

$$\{t \mid \exists s \in \text{section } (t [\text{course_id}] = s [\text{course_id}] \wedge \\ s [\text{semester}] = \text{"Fall"} \wedge s [\text{year}] = 2009 \\ \wedge \exists u \in \text{section } (t [\text{course_id}] = u [\text{course_id}] \wedge \\ u [\text{semester}] = \text{"Spring"} \wedge u [\text{year}] = 2010))\}$$

- Find the set of all courses taught in the Fall 2009 semester, but not in the Spring 2010 semester

$$\{t \mid \exists s \in \text{section } (t [\text{course_id}] = s [\text{course_id}] \wedge \\ s [\text{semester}] = \text{"Fall"} \wedge s [\text{year}] = 2009 \\ \wedge \neg \exists u \in \text{section } (t [\text{course_id}] = u [\text{course_id}] \wedge \\ u [\text{semester}] = \text{"Spring"} \wedge u [\text{year}] = 2010))\}$$



Safety of Expressions

- It is possible to write tuple calculus expressions that generate infinite relations.
- For example, $\{ t \mid \neg t \in r \}$ results in an infinite relation if the domain of any attribute of relation r is infinite
- To guard against the problem, we restrict the set of allowable expressions to safe expressions.
- An expression $\{ t \mid P(t) \}$ in the tuple relational calculus is *safe* if every component of t appears in one of the relations, tuples, or constants that appear in P
 - NOTE: this is more than just a syntax condition.
 - ▶ E.g. $\{ t \mid t[A] = 5 \vee \mathbf{true} \}$ is not safe --- it defines an infinite set with attribute values that do not appear in any relation or tuples or constants in P .



Universal Quantification

- Find all students who have taken all courses offered in the Biology department
 - $\{t \mid \exists r \in \text{student } (t[ID] = r[ID]) \wedge$
 $(\forall u \in \text{course } (u[\text{dept_name}] = \text{"Biology"} \Rightarrow$
 $\exists s \in \text{takes } (t[ID] = s[ID] \wedge$
 $s[\text{course_id}] = u[\text{course_id}]))\}$
 - Note that without the existential quantification on student, the above query would be unsafe if the Biology department has not offered any courses.



Domain Relational Calculus



Domain Relational Calculus

- A nonprocedural query language equivalent in power to the tuple relational calculus
- Each query is an expression of the form:

$$\{ \langle x_1, x_2, \dots, x_n \rangle \mid P(x_1, x_2, \dots, x_n) \}$$

- $x_1, x_2, \dots, x_n$ represent domain variables
- P represents a formula similar to that of the predicate calculus



Example Queries

- Find the *ID*, *name*, *dept_name*, *salary* for instructors whose salary is greater than \$80,000
 - $\{ \langle i, n, d, s \rangle \mid \langle i, n, d, s \rangle \in instructor \wedge s > 80000 \}$
- As in the previous query, but output only the *ID* attribute value
 - $\{ \langle i \rangle \mid \langle i, n, d, s \rangle \in instructor \wedge s > 80000 \}$
- Find the names of all instructors whose department is in the Watson building
 - $\{ \langle n \rangle \mid \exists i, d, s (\langle i, n, d, s \rangle \in instructor \wedge \exists b, a (\langle d, b, a \rangle \in department \wedge b = \text{"Watson"})) \}$





Example Queries

- Find the set of all courses taught in the Fall 2009 semester, or in the Spring 2010 semester, or both

$$\{ \langle c \rangle \mid \exists a, s, y, b, r, t (\langle c, a, s, y, b, t \rangle \in \text{section} \wedge s = \text{"Fall"} \wedge y = 2009) \vee \exists a, s, y, b, r, t (\langle c, a, s, y, b, t \rangle \in \text{section}] \wedge s = \text{"Spring"} \wedge y = 2010)) \}$$

This case can also be written as

$$\{ \langle c \rangle \mid \exists a, s, y, b, r, t (\langle c, a, s, y, b, t \rangle \in \text{section} \wedge ((s = \text{"Fall"} \wedge y = 2009) \vee (s = \text{"Spring"} \wedge y = 2010))) \}$$

- Find the set of all courses taught in the Fall 2009 semester, and in the Spring 2010 semester

$$\{ \langle c \rangle \mid \exists a, s, y, b, r, t (\langle c, a, s, y, b, t \rangle \in \text{section} \wedge s = \text{"Fall"} \wedge y = 2009) \wedge \exists a, s, y, b, r, t (\langle c, a, s, y, b, t \rangle \in \text{section}] \wedge s = \text{"Spring"} \wedge y = 2010)) \}$$



Safety of Expressions

The expression:

$$\{ \langle x_1, x_2, \dots, x_n \rangle \mid P(x_1, x_2, \dots, x_n) \}$$

is safe if all of the following hold:

1. All values that appear in tuples of the expression are values from *dom* (*P*) (that is, the values appear either in *P* or in a tuple of a relation mentioned in *P*).
2. For every “there exists” subformula of the form $\exists x (P_1(x))$, the subformula is true if and only if there is a value of *x* in *dom* (*P*₁) such that *P*₁(*x*) is true.
3. For every “for all” subformula of the form $\forall x (P_1(x))$, the subformula is true if and only if *P*₁(*x*) is true for all values *x* from *dom* (*P*₁).



Universal Quantification

- Find all students who have taken all courses offered in the Biology department
 - $\{ \langle i \rangle \mid \exists n, d, tc (\langle i, n, d, tc \rangle \in \text{student} \wedge$
 $(\forall ci, ti, dn, cr (\langle ci, ti, dn, cr \rangle \in \text{course} \wedge dn = \text{"Biology"} \Rightarrow \exists si, se, y, g (\langle i, ci, si, se, y, g \rangle \in \text{takes}))) \}$
 - Note that without the existential quantification on student, the above query would be unsafe if the Biology department has not offered any courses.

* Above query fixes bug in page 246, last query



End of Chapter 6

Database System Concepts, 6th Ed.

©Silberschatz, Korth and Sudarshan

See www.db-book.com for conditions on re-use



Figure 6.1

<i>ID</i>	<i>name</i>	<i>dept_name</i>	<i>salary</i>
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Califieri	History	62000
76543	Singh	Finance	80000
76766	Click	Biology	72000
83821	Brandt	Comp. Sci.	92000
98345	Kiln	Elec. Eng.	80000



Figure 6.2

<i>ID</i>	<i>name</i>	<i>dept name</i>	<i>salary</i>
22222	Einstein	Physic.s	95000
33456	Gold	Physics	87000



Figure 6.61

<i>ID</i>	<i>name</i>	<i>salary</i>
10101	Srinivasan	65000
12121	Wu	90000
15151	Mozart	40000
22222	Einstein	95000
32343	El Said	60000
33456	Gold	87000
45365	Katz	75000
58583	CalHieri	62000
76343	Singh	80000
76766	Crick	72000
83821	Brandl	92000
98345	Kim	80000



Figure 6.4

<i>course_id</i>	<i>sec_id</i>	<i>semester</i>	<i>year</i>	<i>building</i>	<i>room_number</i>	<i>time_slot_id</i>
BIO-101	1	Summer	2009	Painter	514	B
BTO-301	1	Summer	2010	Painter	514	A
CS-101	1	Fall	2009	Packard	101	H
CS-101	1	Spring	2010	Packard	101	F
CS-190	1	Spring	2009	Taylor	3128	E
CS-190	2	Spring	2009	Taylor	3128	A
CS-315	1	Spring	2010	Watson	120	D
CS-319	1	Spring	2010	Watson	100	B
CS-319	2	Spring	2010	Taylor	3128	C
CS-347	1	Fall	2009	Taylor	3128	A
EE-181	1	Spring	2009	Taylor	3128	C
FIN-201	1	Spring	2010	Packard	101	B
HIS-351	1	Spring	2010	Painter	514	C
MU-199	1	Spring	2010	Packard	101	D
PHY-101	1	Fall	2009	Watson	100	A



Figure 6.5

<i>course id</i>
CS-101
CS-315
CS-319
CS-347
FIN-201
HIS-351
MU-199
PHY-101



Figure 6.64

<u>course id</u>
CS-J47
PTTY-101



Figure 6.07

<i>TO</i>	<i>course_id</i>	<i>Sec_id</i>	<i>Sernester</i>	<i>year</i>
10101	CS-101	1	Fall	2009
10101	CS-315	1	Spring	2010
10101	CS-347	1	Fall	2009
12.12.1	FTN-201	1	Spring	20.10
15151	MU-199	1	Spring	2010
22222	PHY-101	1	Fall	2009
32343	HTS-351	1	Spring	20.10
45565	CS-101	1	Spring	2010
45565	CS-319	1	Spring	2010
76766	BI0-101	1	Summer	2009
76766	BI0-301	1	Summer	2010
83821	CS-190	1	Spring	2009
83821	CS-190	2	Spring	2009
83821	CS-319	2	Spring	2010
98345	EE-131	1	Spring	2009



Figure 6.66

<i>Inst.TD</i>	<i>name</i>	<i>dept_name</i>	<i>salary</i>	<i>teaches.TO</i>	<i>course_id</i>	<i>sec_id</i>	<i>semester</i>	<i>year</i>
10101	Srinivasan	Comp. Sci	65000	10101	CS-101	1	Fall	2009
10101	Srinivasan	Comp. Sci	65000	10101	CS-315	1	Spring	2010
10101	Srinivasan	Comp. Sci	65000	10101	CS-347	1	Fall	2009
10101	Srinivasan	Comp. Sci	65000	12121	FIN-201	1	Spring	2010
10101	Srinivasan	Comp. Sci	65000	15151	MU-199	1	Spring	2010
10101	Srinivasan	Comp. Sci.	65000	22222	PHY-101	1	Fall	2009
...	...	...	...	...	...	...	...	...
12121	Wu	Finance	90000	10101	CS-101	1	Fall	2009
12121	Wu	Finance	90000	10101	CS-315	1	Spring	2010
12121	Wu	Pinance	90000	10101	CS-347	1	Fall	2009
12121	Wu	Pinance	90000	12121	FIN-201	1	Spring	2010
12121	Wu	Finance	90000	15151	MU-199	1	Spring	2010
12121	Wu	Pinance	90000	22222	PHY-101	1	Fall	2009
...	...	...	...	...	...	...	...	...
15151	Mozart	Music	40000	10101	CS-101	1	Fall	2009
15151	Mozart	Music	40000	10101	CS-315	1	Spring	2010
15151	Mozart	Music	40000	10101	CS-347	1	Fall	2009
15151	Mozart	Music	40000	12121	FIN-201	1	Spring	2010
15151	Mozart	Music	40000	15151	MU-199	1	Spring	2010
15151	Mozart	Music	40000	22222	PHY-101	1	Fall	2009
...	...	...	...	...	...	...	...	...
22222	Einstein	Physics	95000	10101	CS-101	1	Fall	2009
22222	Einstein	Physics	95000	10101	CS-315	1	Spring	2010
22222	Einstein	Physics	95000	10101	CS-347	1	Fall	2009
22222	Einstein	Physics	95000	12121	FIN-201	1	Spring	2010
22222	Einstein	Physics	95000	15151	MU-199	1	Spring	2010
22222	Einstein	Physics	95000	22222	PHY-101	1	Fall	2009
...	...	...	...	...	...	...	...	...



Figure 6.67

<i>inst.ID</i>	<i>name</i>	<i>dept_name</i>	<i>salary</i>	<i>teacher.ID</i>	<i>course_id</i>	<i>sec_id</i>	<i>semester</i>	<i>year</i>
22222	Einstein	Physics	95000	10101	CS-437	1	Fall	2009
22222	Einstein	Physics	95000	10101	CS-315	1	Spring	2010
22222	Einstein	Physics	95000	12121	FIN-201	1	Spring	2010
22222	Einstein	Physics	95000	15151	MU-199	1	Spring	2010
22222	Einstein	Physics	95000	22222	PHY-101	1	Fall	2009
22222	Einstein	Physics	95000	32343	HIS-351	1	Spring	2010
...	...	...	...	...	...	...	...	...
...	...	...	...	...	...	...	...	...
33456	Gold	Physics	87000	10101	CS-437	1	Fall	2009
33456	Gold	Physics	87000	10101	CS-315	1	Spring	2010
33456	Gold	Physics	87000	12121	FTN-201	1	Spring	2010
33456	Gold	Physics	87000	15151	MU-199	1	Spring	2010
33456	Gold	Physics	87000	22222	PHY-101	1	Fall	2009
33456	Gold	Physics	87000	32343	HTS-351	1	Spring	2010
...	...	...	...	...	...	...	...	...
...	...	...	...	...	...	...	...	...



Figure 6.10

<i>nam</i>	<i>r.oursl"_id</i>
Einstein	PHY-101



Figure 6.5

<i>salary</i>
6.5000
90000
40000
60000
87000
7.5000
62000
72000
80000
92000



Figure 6.12

salary
95000



Figure 6.13

<i>course_id</i>
CS-101



Figure 6.14

<i>ID</i>	<i>name</i>	<i>dept_name</i>	<i>sal</i>	<i>course_id</i>	<i>sec_id</i>	<i>semester</i>	<i>year</i>
10101	Srinivasan	Comp. Sci	65000	CS-101	1	Fall	2009
10101	Srinivasan	Comp. Sci	65000	CS-315	1	Spring	2010
10101	Srinivasan	Comp. Sci	65000	CS-347	1	Fall	2009
12121	Wu	Finance	90000	FIN-201	1	Spring	2010
15151	Mozali	Music	40000	MU-199	1	Spring	2010
22222	Einstein	Physics	95000	PHY-101	1	Fall	2009
32343	El Said	History	60000	HIS-351	1	Spring	2010
45565	Katz	Comp. Sci.	75000	CS-101	1	Spring	2010
45565	Katz	Comp. Sci.	75000	CS-319	1	Spring	2010
76766	Crick	Biology	72000	BIO-101	1	Summer	2009
76766	Crick	Biology	72000	BIO-301	1	Summer	2010
83821	Brandt	Comp. Sci.	92000	CS-190	1	Spring	2009
83821	Brandt	Comp. Sci.	92000	CS-190	2	Spring	2009
83821	Brandt	Comp. Sci.	92000	CS-319	2	Spring	2010
98345	Kim	Elec. Eng.	80000	EE-481	1	Spring	2009



Figure 6.15

<i>name</i>	<i>course_id</i>
Sriniv<Jsan	CS-101
Sriniv<Jsan	CS-315
Sriniv<Jsan	CS-347
\Nu	FIN-201
Moz<lrt	MU-199
Einstein	PHY-101
Fl Suđ	HIS-351
Katz	CS-101
Katz	CS-319
Crick	BIO-101
Crick	BIO-301
Brandt	CS-190
Brandt	CS-319
Kim	F.F>"18đ



Figure 6.16

<i>nanIt</i>	<i>lilie</i>
Brandt	Game Design
Brandt	Image Processing
Katz	Image Processing
Katz	Tntro. to Computer Sdcnc.e
Srinivasan	Tntro. to Computer Sdcnc.e
Srinivasan	Robotics
Srinivasan	Database System Concepts



Figure 6.17

ID	name	dept_name	salary	course_id	sec_id	semester	year
10101	Srinivasan	Comp. Sci.	65000	cs-101	1	Fall	2009
10101	Srinivasan	Comp. Sci.	65000	CS-315	1	Spring	2010
10101	Srinivasan	Comp. Sci.	65000	CS-347	1	Fall	2009
12121	Wu	Finance	90000	FTN-201	1	Spring	2010
15151	Mozart	Music	40000	MU-199	1	Spring	2010
22222	Einstein	Physics	95000	PHY-101	1	fall	2009
32343	El Said	History	60000	HIS-351	1	Spring	2010
33456	Gold	Physics	87000	null	null	null	null
45565	Katz	Comp. Sci.	75000	CS-101	1	Spring	2010
45565	Katz	Comp. Sci.	75000	CS-319	1	Spring	2010
58583	Califieri	History	62000	null	null	null	null
76543	Singh	Finance	80000	null	null	null	null
76766	Crick	Biology	72000	BIO-101	1	Summer	2009
76766	Gick	Biology	72000	BIO-301	1	Summer	2010
83821	Brandt	Comp. Sci.	92000	cs-190	1	Spring	2009
83821	Brandt	Comp. Sci.	92000	CS-190	2	Spring	2009
83821	Brandt	Comp. Sci.	92000	CS-319	2	Spring	2010
98345	Kim	Elec. Eng.	80000	EE-181	1	Spring	2009



Figure 6.18

<i>JD</i>	<i>course_id</i>	<i>sec_id</i>	<i>semester</i>	<i>year</i>	<i>name</i>	<i>dept_name</i>	<i>salary</i>
10101	CS-315	1	Spring	2010	Srinivasan	Comp. Sci.	65000
10101	CS-347	1	Fall	2009	Srinivasan	Comp. Sci.	65000
12121	FTN-201	1	Spring	2010	Wu	Finance	90000
15151	JMU-199	1	Spring	2010	Mozart	Music	40000
22222	PHY-101	1	Fall	2009	Einstein	Physics	95000
32343	HIS-351	1	Spring	2010	El Said	History	60000
33456	<i>null</i>	<i>null</i>	<i>null</i>	<i>null</i>	Gold	Physics	87000
45565	CS-101	1	Spring	2010	Katz	Comp. Sci.	75000
45565	CS-319	1	Spring	2010	Katz	Comp. Sci.	75000
58583	<i>null</i>	<i>null</i>	<i>null</i>	<i>null</i>	Califiori	History	62000
76543	<i>null</i>	<i>null</i>	<i>null</i>	<i>null</i>	Singh	finance	80000
76766	BI0-101	1	Summer	2009	Crick	Biology	72000
76766	BI0-301	1	Summer	2010	Crick	Biology	72000
83821	CS-190	1	Spring	2009	Brandt	Comp. Sci.	92000
83821	CS-190	2	Spring	2009	Brandt	Comp. Sci.	92000
83821	CS-319	2	Spring	2010	Brandt	Comp. Sci.	92000
98345	EE-181	1	Spring	2009	Kim	Elec. Eng.	80000



Figure 6.19

<i>ID</i>	<i>name</i>	<i>dept name</i>	<i>salary</i>
76766	Crick	Biology	72000
45565	Katz	Comp. Sci.	75000
10101	Srinivasan	Comp. Sci.	65000
83821	Brandl	Comp. Sci.	92000
98345	Kim	Elect. Eng.	80000
12121	Vu	Finance	90000
76543	Singh	Finance	80000
32343	El Said	History	60000
58583	Calituri	History	62000
15151	Mozart	Music	40000
33456	Gold	Physics	81000
22222	Einstein	Physics	95000



Figure 6.20

<i>dept name</i>	<i>salan1</i>
Biology	72000
Comp. Sd.	77333
Elec. Eng.	80000
Finan(;e	85000
Hislory	61000
Musk	40000
Physics	91000



Figure 6.21

<i>nmne</i>
Einstein
Crick
Gold



Deletion

- A delete request is expressed similarly to a query, except instead of displaying tuples to the user, the selected tuples are removed from the database.
- Can delete only whole tuples; cannot delete values on only particular attributes
- A deletion is expressed in relational algebra by:

$$r \leftarrow r - E$$

where r is a relation and E is a relational algebra query.



Deletion Examples

- Delete all account records in the Perryridge branch.

$account \leftarrow account - \sigma_{branch_name = "Perryridge"}(account)$

- Delete all loan records with amount in the range of 0 to 50

$loan \leftarrow loan - \sigma_{amount \geq 0 \text{ and } amount \leq 50}(loan)$

- Delete all accounts at branches located in Needham.

$r_1 \leftarrow \sigma_{branch_city = "Needham"}(account \bowtie branch)$

$r_2 \leftarrow \Pi_{account_number, branch_name, balance}(r_1)$

$r_3 \leftarrow \Pi_{customer_name, account_number}(r_2 \bowtie depositor)$

$account \leftarrow account - r_2$

$depositor \leftarrow depositor - r_3$



Insertion

- To insert data into a relation, we either:
 - specify a tuple to be inserted
 - write a query whose result is a set of tuples to be inserted
- in relational algebra, an insertion is expressed by:

$$r \leftarrow r \cup E$$

where r is a relation and E is a relational algebra expression.

- The insertion of a single tuple is expressed by letting E be a constant relation containing one tuple.



Insertion Examples

- Insert information in the database specifying that Smith has \$1200 in account A-973 at the Perryridge branch.

$$account \leftarrow account \cup \{("A-973", "Perryridge", 1200)\}$$
$$depositor \leftarrow depositor \cup \{("Smith", "A-973")\}$$

- Provide as a gift for all loan customers in the Perryridge branch, a \$200 savings account. Let the loan number serve as the account number for the new savings account.

$$r_1 \leftarrow (\sigma_{branch_name = "Perryridge"}(borrower \bowtie loan))$$
$$account \leftarrow account \cup \Pi_{loan_number, branch_name, 200}(r_1)$$
$$depositor \leftarrow depositor \cup \Pi_{customer_name, loan_number}(r_1)$$



Updating

- A mechanism to change a value in a tuple without changing *all* values in the tuple
- Use the generalized projection operator to do this task

$$r \leftarrow \Pi_{F_1, F_2, K, F_l}(r)$$

- Each F_i is either
 - the i^{th} attribute of r , if the i^{th} attribute is not updated, or,
 - if the attribute is to be updated F_i is an expression, involving only constants and the attributes of r , which gives the new value for the attribute



Update Examples

- Make interest payments by increasing all balances by 5 percent.

$account \leftarrow \Pi_{account_number, branch_name, balance * 1.05} (account)$

- Pay all accounts with balances over \$10,000 6 percent interest and pay all others 5 percent

$account \leftarrow \Pi_{account_number, branch_name, balance * 1.06} (\sigma_{BAL > 10000} (account)) \cup \Pi_{account_number, branch_name, balance * 1.05} (\sigma_{BAL \leq 10000} (account))$



Example Queries

- Find the names of all customers who have a loan and an account at bank.

$$\Pi_{customer_name} (borrower) \cap \Pi_{customer_name} (depositor)$$

- Find the name of all customers who have a loan at the bank and the loan amount

$$\Pi_{customer_name, loan_number, amount} (borrower \bowtie loan)$$



Example Queries

- Find all customers who have an account from at least the “Downtown” and the Uptown” branches.

- Query 1

$$\Pi_{customer_name} (\sigma_{branch_name = \text{“Downtown”}} (depositor \bowtie account)) \cap \\ \Pi_{customer_name} (\sigma_{branch_name = \text{“Uptown”}} (depositor \bowtie account))$$

- Query 2

$$\Pi_{customer_name, branch_name} (depositor \bowtie account) \\ \div \rho_{temp(branch_name)} (\{(\text{“Downtown”}), (\text{“Uptown”})\})$$

Note that Query 2 uses a constant relation.



Bank Example Queries

- Find all customers who have an account at all branches located in Brooklyn city.

$$\begin{aligned} & \Pi_{customer_name, branch_name} (depositor \bowtie account) \\ & \div \Pi_{branch_name} (\sigma_{branch_city = \text{"Brooklyn"}} (branch)) \end{aligned}$$